

Hashing und Hashmaps - Kontrollfragen



Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de, Raum 149B
Version: 1.0.0

1. Grundlagen des Hashings



Motivation und Grundidee

1.1. Warum überhaupt Hashing?

Eine (duplikatfreie) Symboltabelle lässt sich u. a. über eine verkettete Liste, ein geordnetes Array oder einen binären Suchbaum realisieren. Welches Ziel verfolgt Hashing? Was ist der Preis?

1.2. Grundidee und Begriff der Hashfunktion

Beschreiben Sie die Grundidee des Hashings. Was leistet eine Hashfunktion in diesem Kontext?



Herausforderungen und Kompromiss

1.3. Welche Probleme müssen beim Hashing gelöst werden?

1.4. Klassischer Kompromiss zwischen Raum und Zeit

Hashing ist ein Kompromiss zwischen Speicherplatz und Laufzeit. Skizzieren Sie die beiden Extremfälle.

Berechnung der Hashfunktion

1.5. Idealistisches Ziel und die Wahl der Ziffern

Was ist das *idealistische Ziel* bei der Berechnung einer Hashfunktion?

2. Hashfunktionen



Definition und Eigenschaften

2.1. Definition

Was bedeutet die Bedingung $|M| \geq |\mathbb{Z}_n|_{[1]}$, und welche Konsequenz folgt daraus unmittelbar?

2.2. Surjektiv und gleichverteilt

Was bedeutet es, dass eine Hashfunktion *surjektiv* bzw. *gleichverteilt* ist?

[1] M ist die Menge der Schlüssel.



Konstruktion: Zeichenketten (Horner)

2.3. Horner-Methode

Warum verwendet Horners Methode zur Berechnung des Hashes einer Zeichenkette den Faktor 31?



Konstruktion: Modulo und Multiplikation

2.4. Modulo-Hashfunktion und Primzahlen

Warum sollte n bei der Modulo-Hashfunktion möglichst eine Primzahl sein?

2.5. Periodische Schlüssel

Ihre Schlüssel kommen mit einer festen Schrittweite a (z. B. 12).
Sie verwenden $h(x) = x \bmod n$. Wie wählen Sie n sinnvoll?

3. Hashing in Programmiersprachen



Der hashCode/equals-Vertrag

3.1. Muss vs. soll

Welche Regel müssen `__hash__` / `__eq__` (Python) bzw. `hashCode()` / `equals(Object)` (Java) **zwingend** erfüllen, welche ist nur **wünschenswert**?

3.2. Konstante Hashfunktion

Eine Klasse gibt in `hashCode()` / `__hash__` immer 1 zurück. Ist das korrekt? Welche Folgen hat es?

3.3. Unveränderlichkeit nach dem Einfügen

Warum darf ein Objekt, *nachdem* es in einem Set/Dictionary gespeichert wurde, hinsichtlich der hash-relevanten Felder nicht mehr verändert werden? Was passiert sonst konkret?

3.4. Python's `hash()` ändert sich

In Python liefert `hash("...")` für denselben String nach jedem Neustart der Umgebung einen *anderen* Wert. Warum — und was folgt daraus?

4. Hashtabellen



Vom Hashwert zum Tabellenindex

4.1. Zweistufige Indexberechnung

Bei der konkreten Verwendung entsteht der Tabellenindex in **zwei**

Stufen: zuerst die Hashfunktion, dann der eigentliche Index.

- Beschreiben Sie beide Stufen und ordnen Sie zu, wer jeweils zuständig ist (Schlüsseltyp vs. Datenstruktur).
- Was passiert mit dem Hashwert und mit dem Index, wenn die Tabelle vergrößert wird?
- Warum ist diese Trennung sinnvoll?

4.2. Belegungsfaktor

Was ist der Belegungsfaktor α einer Hashtabelle, und warum ist er für die Effizienz entscheidend?



Kollisionsauflösung

4.3. Nennen Sie die zwei grundsätzlichen Strategien zur Kollisionsauflösung und ihren Kernunterschied. Welche erlaubt $\alpha > 1$?

4.4. Separate vs. direkte Verkettung

Worin unterscheiden sich separate und direkte Verkettung?



Offene Adressierung

4.5. Sondieren kann scheitern

Welche Sondierungsfunktion kann scheitern und somit für einen Wert **kein** freier Platz gefunden werden, obwohl die Tabelle nicht voll ist?

4.6. Einfügen mit linearem Sondieren

Fügen Sie in eine leere Hashtabelle mit $n = 7$ und h_7^{mod} nacheinander die Werte 8, 15, 22 mit **linearem Sondieren** ein. Wie sieht die Tabelle aus?

4.7. Löschen bei offener Adressierung

Beim Löschen eines Elements aus einer Hashtabelle mit offener Adressierung darf man den Platz **nicht** einfach auf `frei`/`nil` setzen. Warum nicht — und wie löst man das Problem? Was ist dabei zu beachten?

5. Angriffe auf algorithmische Komplexität



Hash-Kollisions-Angriffe

5.1. Denial of Service über Hashkollisionen

Erläutern Sie am Beispiel von Java's `String.hashCode()`, wie ein Angriff auf die algorithmische Komplexität funktioniert und wie man sich dagegen schützt.